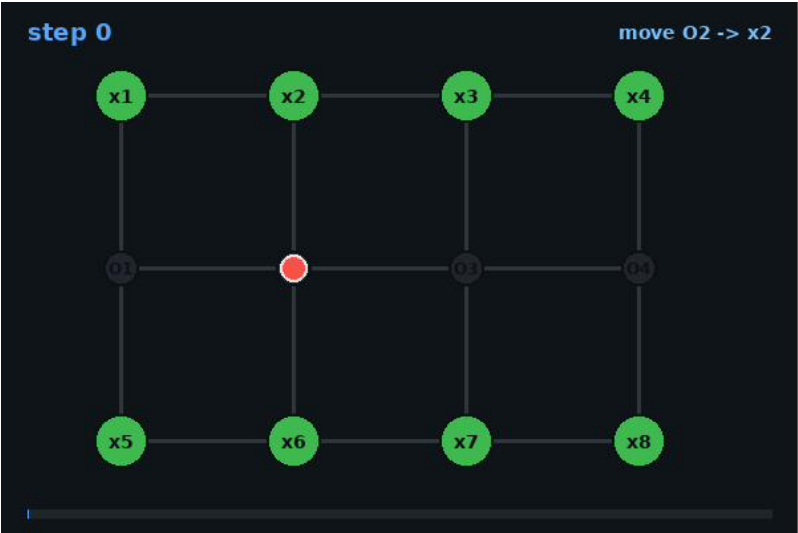


Agricultural Robotics — Crop Monitoring

Assignment D5-V1 · PDDL & PDDL+ Planning Project

Artificial Intelligence for Robotics II

A single mobile robot patrols a field of crop plots, inspecting each plot's condition and reporting its observations. The field is a graph of plots joined by routing junctions, with the robot starting at O2. The figure below shows this field as drawn by the plan visualiser.



The monitoring field as rendered by the plan visualiser: green nodes are crop plots (x1–x8); faded nodes (O1–O4) are routing-only junctions; the red dot is the robot, starting at junction O2.

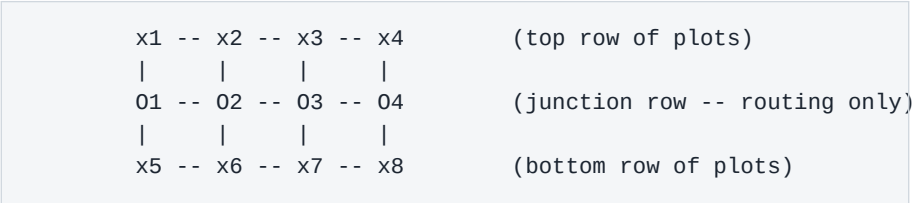
Stage	Formalism	Core challenge
Q1	Classical PDDL + numeric cost	Visit and report all plots with the fewest moves
Q2	PDDL+ (process + events)	Inspect every plot before its crop degrades to failure

A single robot monitors a field of crop plots: it inspects each plot's condition and reports the observation. The scenario is modelled at two levels — static routing (Q1) and a race against autonomous crop degradation (Q2).

1. Domain and Planning Problem

A mobile robot operates in an agricultural field divided into plots. Each plot contains crops that must be monitored. Monitoring has two parts: the robot **inspects** a plot (it must physically be at the plot to read the crop's condition) and then **reports** the observation (logged once the plot has been inspected).

The field is modelled as a **graph**. Crop plots (x1...x8) are connected through intermediate **junction** nodes (01...04) that the robot can travel through but does not inspect. The robot starts at junction 02 and moves edge by edge.



Field layout (plots, junctions, and edges).

Two levels of the same scenario

The project models one consistent scenario at two levels of expressiveness. The conceptual jump from Q1 to Q2 is **time**: in Q1 the world is static and only changes when the robot acts; in Q2 the crops degrade on their own, so “doing nothing” has a cost and the **timing** of actions determines success.

- **Q1 — Classical PDDL (no time)**: monitoring framed as a routing problem — visit and report every plot using the fewest moves.
- **Q2 — PDDL+ (continuous time)**: monitoring framed as a *race* against crops that degrade autonomously; each plot must be reported before it crosses a failure threshold.

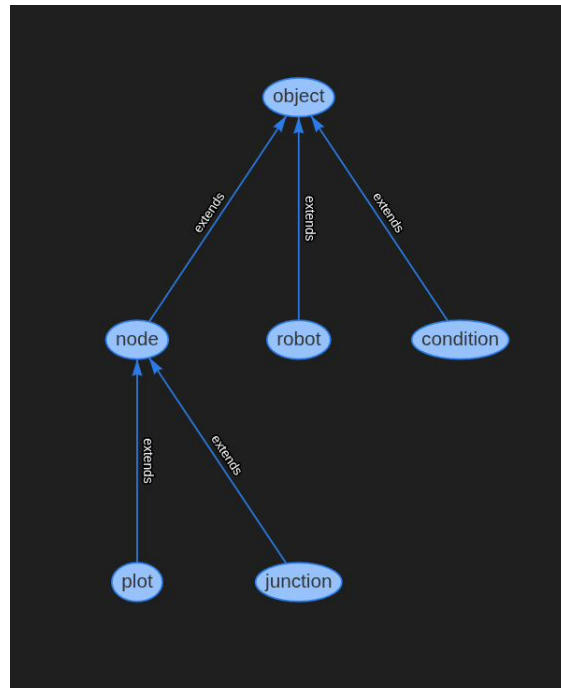
Planning goal (both stages)

Every plot must end up **reported**: (reported x1) ... (reported x8). In Q2 the additional implicit requirement is that no plot crosses its failure threshold before being reported.

2. Modelling Choices and Rationale

2.1 Common structure (shared by Q1 and Q2)

Both domains share a typed object model. The type hierarchy splits every position the robot can occupy into **plot** (inspectable) and **junction** (routing-only), both deriving from a common node type.



Type hierarchy. node splits into plot and junction; robot and condition are independent types. All derive from object.

Core predicates shared by both models:

- (robot-at ?r ?n) — robot position on a node.
- (connected ?n1 ?n2) — graph edges (declared per problem).
- (crop-condition ?p ?c) — symbolic crop state.
- (inspected ?p) — the robot has observed this plot.
- (reported ?p) — the observation has been logged.

Why these choices:

- **Plots vs. junctions are separate types** so move can traverse any node, but only plots can be inspected. This avoids duplicating actions and keeps the graph clean.
- **Inspection is passive** — it only sets (inspected ?p) and never changes the crop. Monitoring observes; it does not intervene.
- **Reporting is decoupled from position** — report only requires that the plot was inspected, so a reading can be logged from anywhere; the robot must, however, not be moving while it reports.

2.2 Q1 — Classical PDDL

Classical PDDL has **no notion of time**: the world only changes through actions, and crop conditions are static. Q1 uses three instantaneous actions and a single numeric cost fluent.

Action	Precondition	Effect
move ?r ?from ?to	robot at ?from; edge connected	robot moves; total-cost += 1
inspect ?r ?p	robot at ?p; not inspected	marks plot inspected
report ?r ?p	?p inspected; not reported	marks plot reported

Cost model. A single numeric fluent (`total-cost`) increases **only on move**. This turns the objective into a routing problem: visit and report all plots while moving as little as possible. With an optimal planner and a minimise-cost metric, the planner returns the fewest-moves monitoring tour, showing the model has real planning depth — the route, not just the action set, matters.

A strategy that did NOT work (and why)

An earlier idea made the robot visit critical plots first using a *state-dependent* move cost: start a counter at the sum of all priorities, and have each move add the current counter while each report decreased it. In practice **ENHSP could not optimise this cost** — its optimal search reasons about uniform (unit) action costs and effectively ignores a per-action cost that varies with the state. We therefore kept the simple, uniform +1-per-move cost, which ENHSP optimises correctly. Priority handling is instead expressed through the timing pressure in Q2.

2.3 Q2 — PDDL+ (numeric fluents, a process, and events)

PDDL+ adds **continuous time** through `:process` and `:event`. Crops now degrade **autonomously**, independent of the robot.

Numeric fluents. Q2 introduces continuous quantities that classical predicates cannot express:

- `(condition-level ?p)` — the crop's health, decreasing over time.
- `(degrade-rate ?p)` — how fast each plot degrades.
- `(move-timer ?r) / (report-timer ?r)` — drive action durations.

The degradation process. A single process, `degrade`, runs continuously on every plot whose level is still above zero, lowering `condition-level` at that plot's `degrade-rate`. This is the heart of Q2: the crop loses health smoothly over time whether or not the robot is present, so inactivity is no longer free.

Events. The `crop-fails` event is the failure condition: if a crop crosses the threshold (`level ≤ 20`) **before it has been reported**, it is permanently marked failed and the goal becomes unreachable. Two further events, `move-complete` and `report-complete`, close out the timed actions.

Key design point — a durative action replaced by one process + two events

ENHSP does **not** support `:durative-action`. A timed action such as `move` (which would naturally be a 1-second durative action) is therefore decomposed into three PDDL+ pieces:

- an instantaneous **start-move** action that begins the motion and sets the robot "moving";
- a **moving process** that advances `move-timer` continuously while the robot is in motion;
- a **move-complete event** that fires the instant `move-timer ≥ 1`, teleporting the robot to its destination and resetting the timer.

The same pattern (start-action + timer process + completion event) gives `report` its 0.2-second duration. In short: **one durative action is emulated by a start action, one process, and a completion event** — and the failure behaviour adds the second event, `crop-fails`.

3. Plan Walkthrough and Expected Behaviour

How to read the plans

The planner only prints a **list of timed actions**, which on its own is not very readable. It is strongly recommended to view the corresponding **GIF animations in the README**, where the field, the robot's path, inspections, reports, and (in Q2) the shrinking crop-condition rings are shown visually. The walkthroughs below interpret what each plan does, step by step.

Q1 — Problem 1: stable crops

All eight plots are good; the robot starts at O2; no metric is set, so any valid plan is accepted. Interpreting the plan:

1. The robot leaves O2 and moves to an adjacent plot (e.g. O2→x2), one edge per move action.
2. At each plot it performs an **inspect**, which records the crop condition for that plot.
3. It then performs a **report** for that plot; because reporting is position-independent, reports can be interleaved freely.
4. It repeats move–inspect–report across all eight plots, routing through junctions where needed.
5. The plan ends once every plot is reported. This confirms the model is solvable and correct; since no metric is set, the route is not necessarily minimal.

Behaves as expected: a complete, valid monitoring sweep.

Q1 — Problem 2: cost-optimized

Same field and goal, but a minimise-cost metric with the optimal planner is now used. Interpreting the plan:

1. Because each move costs +1 and inspect/report are free, the planner searches for the route that touches all eight plots with the fewest edges.
2. Reports are scheduled opportunistically (they cost nothing and do not require being at the plot), so the optimiser only has to minimise travel.
3. The resulting action list contains **fewer move actions** than Problem 1's plan — the visible payoff of optimization.

Behaves as expected: a fewest-moves monitoring tour.

Q2 — Problem 1: timing irrelevant

All plots start at condition-level 90 with a slow degrade-rate 2. Time to failure is $(90 - (-20)) / 2 = 55$ s, while the robot finishes everything in roughly 20 seconds. Interpreting the plan:

1. Each **start-move** sets the robot moving; the moving process advances the move-timer; the **move-complete** event fires after 1 second and places the robot at the next node.
2. At each plot the robot inspects, then a **start-report** begins the 0.2-second report, completed by the **report-complete** event.
3. Meanwhile the degrade process is lowering every crop's level, but only gently — no plot comes anywhere near the failure threshold.
4. Every plot is reported well within the safe window, so the visiting order does not matter.

Behaves as expected: the PDDL+ model is correct in the non-urgent case; timing has no effect on feasibility here.

Q2 — Problem 2: timing critical

Degradation is now mixed. Plot x8 starts at 70 with rate 10, so it fails in $(70 - 20) / 10 = 5$ s; x3 is also critical. Interpreting the plan:

1. Reaching x8 from O2 needs at least three moves (O2→O3→O4→x8 = 3 seconds on the move timer), so the robot has very little slack before x8's crop-fails event would trigger.
2. The planner therefore **prioritises the fast-degrading plots first**, driving the shortest path to x8, inspecting and reporting it before its level reaches 20.
3. It then handles the other critical plot (x3) and sweeps the remaining slow-degrading plots, whose generous time-to-failure leaves room to finish.
4. A naive route that ignored x8's rate would reach it too late; the crop-fails event would mark x8 failed and the goal would become unreachable.

Behaves as expected: here the *schedule* of inspections, not merely their existence, determines feasibility — the central PDDL+ lesson that inactivity has a cost.

4. Limitations and Known Issues

- **Discrete approximation of a continuous quantity (Q1).** Classical PDDL has no time, so in Q1 crop state is a static symbolic label. The “crops change over time” requirement is only honoured in Q2; Q1 is a routing abstraction of the same field.
- **State-dependent cost is not optimised by ENHSP.** As noted in section 2.2, a priority-weighted move cost could not be optimised, so Q1 uses a uniform per-move cost.
- **Durative actions are unavailable in ENHSP.** Action durations are emulated with timer fluents, a process, and completion events. This works but inflates the model with extra movement/reporting state and events compared with a native durative-action formulation.
- **Time-step sensitivity.** PDDL+ planning discretises time with the planning/execution deltas (0.1 here). Too coarse a delta can miss the instant an event should fire; too fine a delta slows search. The Problem 2 thresholds were chosen with margin so the result is robust to the step.
- **Single robot, fixed topology.** The model assumes one robot and a static graph; it does not capture multi-robot coordination, dynamic obstacles, or repeated monitoring cycles.
- **Failure is permanent and binary.** A crop either fails at the threshold or it does not; there is no partial-quality or recovery model.

Net assessment

Within the stated scope the four plans behave exactly as intended: Q1 demonstrates correct routing and cost-optimal touring, and Q2 demonstrates autonomous degradation, a working failure event, and a feasibility result that genuinely depends on action timing.

5. Discussion

A. Passive observation vs. intervention

In this assignment the robot is a pure **monitor**: its only job is to **inspect and report** each plot before that plot's crop level drops below the failure threshold. The model deliberately assumes that any **intervention is performed by a separate entity** that receives the report — an irrigation system, an agronomist, or a second robot. The monitoring robot's responsibility ends at delivering the observation in time; what happens to the crop afterwards is outside its scope. This is why inspection is modelled as **passive**: it reads the crop state but never changes it.

If instead the same robot were allowed to **intervene**, the problem would change substantially:

- **The mission goal changes.** It would no longer be enough to report before failure; the robot would have to **complete the intervention before the failure threshold**, turning a reporting deadline into a treatment deadline.
- **Resource management becomes central.** Intervention consumes resources — for example a limited on-board water store that must be applied in time, plus other supplies (fertiliser, pesticide) that may be stored at **different locations** the robot has to detour to and refill from. The plan must then balance routing, deadlines, and stock levels at once.
- **Rehabilitation time appears.** Treatment is not instantaneous: after applying water or nutrients there is a recovery period before the crop level rises, and a treated crop can **degrade again later**, so the robot may need to revisit plots in repeated cycles.

In other words, passive monitoring keeps the problem as a **timed routing-and-reporting** task, whereas adding intervention turns it into a far richer **resource-constrained, deadline-driven scheduling** problem. The current model intentionally stays on the monitoring side of that line, which is exactly what the D5-V1 scenario asks for.

B. Limitations of discrete crop modelling

Q1 represents the crop with a small set of **discrete symbolic labels** (good / normal / critical). This keeps the model fully classical, but loses information in several ways:

- **Loss of resolution.** Real crop health is a continuum; collapsing it into three buckets hides how close a 'normal' plot is to becoming 'critical'.
- **No notion of rate.** A discrete label cannot express *how fast* a crop is declining, so the planner cannot reason about which plot fails first — the very reasoning Q2 Problem 2 depends on.
- **Arbitrary thresholds.** Bucket boundaries are chosen by the modeller, so two plots either side of a boundary look a whole category apart even when nearly identical.
- **Transitions must be hand-authored.** In pure PDDL a discrete state only changes through an explicit action, so the world cannot drift on its own — conflicting with the scenario's requirement that crops change over time without intervention. This is exactly why Q2 moves to a continuous numeric level driven by a process.

Q2's continuous condition-level with a degrade-rate fixes most of this by capturing both the current value and its trajectory, at the cost of added complexity and sensitivity to the time-discretisation step (section 4).

6. Output — How to Reproduce

All plans were produced with a locally built ENHSP planner. (The online solver at solver.planning.domains was avoided — its ENHSP instance is broken and silently returns empty plans.) Each problem is run by pointing ENHSP at the matching domain and problem file with the appropriate planner mode:

Plan	Domain / Problem	Planner mode
Q1 — P1 (stable)	domain1.pddl / problem1_1.pddl	sat-hmrp (satisficing)
Q1 — P2 (cost-opt)	domain1.pddl / problem1_2.pddl	opt-hrmax (optimal, respects metric)
Q2 — P1 (timing OK)	domain2.pddl / problem2_1.pddl	sat-hadd -dp 0.1 -de 0.1
Q2 — P2 (timing crit.)	domain2.pddl / problem2_2.pddl	sat-hadd -dp 0.1 -de 0.1

Planner / flag notes. `sat - *` planners are satisficing — they find a valid plan fast and ignore cost metrics. `opt-hrmax` is optimal — it respects the minimise-cost metric and is used for Q1 Problem 2. The `-dp 0.1 -de 0.1` flags set the planning and execution time deltas, needed in Q2 so the planner steps finely enough to hit the process/event thresholds.

Project files.

File	Role
domain1.pddl	Q1 classical domain (move / inspect / report, total-cost)
problem1_1.pddl	Q1 Problem 1 — stable crops, no metric
problem1_2.pddl	Q1 Problem 2 — cost-optimized tour
domain2.pddl	Q2 PDDL+ domain (numeric fluents, process, events)
problem2_1.pddl	Q2 Problem 1 — timing irrelevant
problem2_2.pddl	Q2 Problem 2 — timing critical
plan_animation_q*.gif	Visualised plan executions (see README)
README.md	Full project write-up

Assignment D5-V1 — Agricultural Robotics: Crop Monitoring.